Appendix of HA Pol-NAS

A. Structures of partial candidate operations.

Here we provide the structures of some candidate operations, including GhostNet bottlenecks [1], ShuffleNetV2 units [2], FasterNet blocks [3], as shown in Fig. 1-Fig. 3.

BN denotes batch normalization, and ReLU is the activation function. $*$ denotes the convolution operation. $\Phi_1; \Phi_2; \cdots; \Phi_k$ are inexpensive operations such as 3×3 depthwise convolutions. The channel split equally divides the tensor into two tensors with half of the original number of channels, and concat concatenates them based on the channel dimension. Channel shuffling reorders the channels. DW Conv denotes depthwise convolution. *Hei*, *Wid*, *C* and *k* represent the height, width, the number of channels and the kernel size, respectively. The remaining operations are commonly used operations, and their structures are not exhibited in detail.

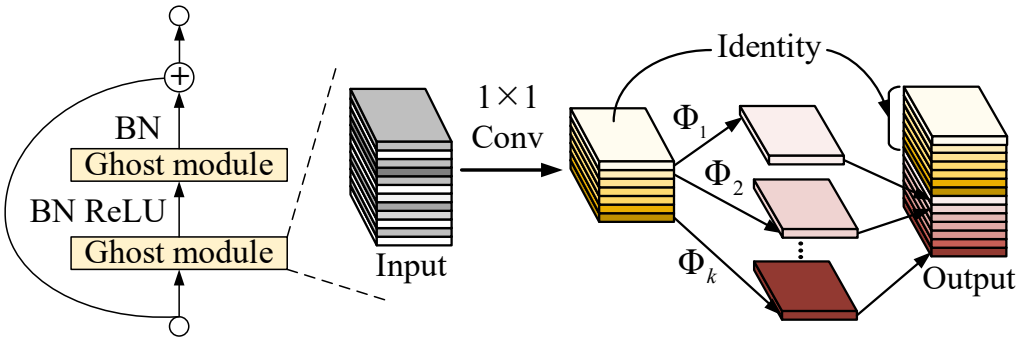

Fig. 1. The structure of the GhostNet bottleneck.

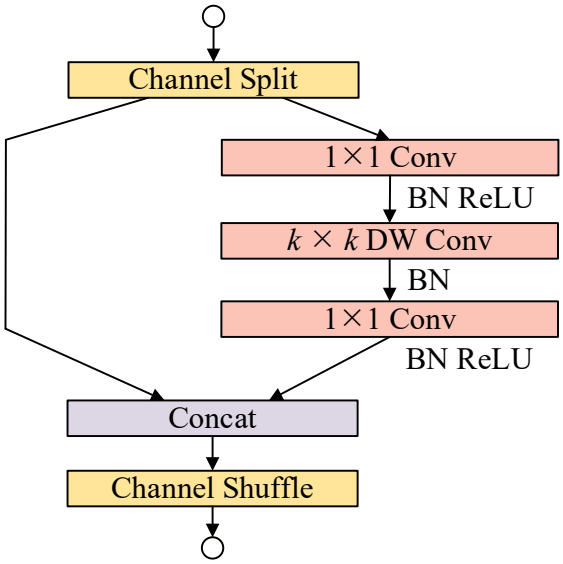

Fig. 2. The structure of the ShuffleNetV2 unit.

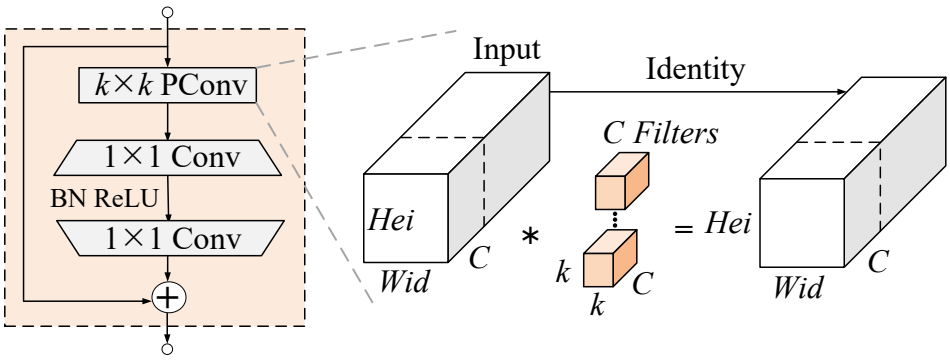

Fig. 3. The structure of the FasterNet block.

B. Explanation of how search time and latency in Tables IV-VII are obtained

The **search time** refers to the actual time consumed for searching the architecture by each method in the search stage.

The **latency** denotes the **measured inference latency** of the searched architecture on real hardware, rather than a precomputed offline value.

The **latency lookup table** is used in the search stage. We first measure and record the inference latency of **each individual operation type** (not the latency of the entire model) in advance. This lookup table is then used in the latency loss to **approximate the overall inference latency** of the model, which effectively reduces the time cost in the search stage.

In this way, we can avoid measuring the inference latency of each candidate model individually in the search stage, which would introduce heavy additional computational overhead.

C. Full configurations of search and training stages

(1) Search stage

The maximum number of layers is 12, and each cell contains 19 candidate operations (i.e., $|\mathcal{O}| = 19$). The output tensor channel of a cell with a downsampling rate of s is $Multi \times s/2$. The capacity of the model is governed by the *Multi* parameter, with a default value of 6. For connections between cells with different downsampling rates s, such as $\frac{s}{2} \rightarrow s$ and $s \rightarrow \frac{s}{2}$, bilinear interpolation is used to halve/double the sizes of the corresponding tensors if necessary, and a 1×1 convolution is utilized to double/halve their channels. The atrous spatial pyramid pooling (ASPP) module used in HA Pol-NAS has three branches, namely, a 1×1 convolution branch, a 3×3 convolution branch with an atrous rate of $48/s$ and an image pooling branch that is identical to that employed in [4]. Each branch involved in ASPP outputs a tensor with $Multi \times s/2$ channels.

The number of epochs in the search stage $Epoch_1$ is set to 40. We choose to update the weights of HA Pol-NAS only in the first 10 epochs ($1 \sim Epoch_2$) because we have observed that the architecture may converge to local optima if its weights are not properly optimized. For the following 30 epochs ($Epoch_2 \sim Epoch_1$), the weights are updated in each epoch, and the architecture parameters are updated for an epoch every 5 epochs (i.e., $Epoch_3 = 5$). When the architecture parameters are optimized, we choose

to optimize the weights for one epoch, followed by optimizing the architecture parameters for another epoch. The balance factor λ is set to 0.1. The weights are fine-tuned using stochastic gradient descent (SGD) with a cosine annealing learning rate schedule, starting from 0.025 and decreasing to 0.001, along with a momentum of 0.9 and a weight decay of 0.0003. The architecture parameters are updated using the Adam optimization algorithm with a learning rate of 0.001, ρ=(0.9, 0.99), and a weight decay rate of 0.001. The batch size is configured to 16. Data augmentation is achieved for D_{trainA} and D_{trainB} through horizontal and vertical flip.

(2) Training stage

Multi is set to 8, which is distinct from its configuration in the search stage. Because the architecture search process is accomplished in the search stage and we should train the model until convergence in the training stage, the ratio of D_{trainA} and D_{trainB} is adjusted from 1:1 to 9:1. Therefore, D_{trainA} is utilized for updating the weights, and D_{trainB} is used for selecting the best model during training. The training epoch $Epoch_4$ is set to 100, and the batch size is 16. The weights are updated via SGD with a learning rate of 0.001, a momentum of 0.9, and a weight decay of 0.0001. Only D_{trainA} is augmented via horizontal and vertical flip. As with Pol-NAS, HA Pol-NAS also adopts an encoder–decoder structure [5]. The difference is that the searched architecture of the encoder is different.

References:

[1] K. Han, Y. Wang, Q. Tian, J. Guo, C. Xu, and C. Xu, "GhostNet: More features from cheap operations," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 1580–1589.

[2] N. Ma, X. Zhang, H.-T. Zheng, and J. Sun, "ShuffleNet v2: Practical guidelines for efficient cnn architecture design," in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018, pp. 116–131.

[3] J. Chen, S.-h. Kao, H. He, W. Zhuo, S. Wen, C.-H. Lee, and S.- H. G. Chan, "Run, don't walk: chasing higher FLOPS for faster neural networks," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 12021–12031.

[4] L.-C. Chen, G. Papandreou, F. Schroff, and H. Adam, "Rethinking atrous convolution for semantic image segmentation," *arXiv preprint arXiv:1706.05587*, 2017.

[5] G. Liu, Y. Li, Y. Chen, R. Shang, and L. Jiao, "Pol-NAS: A neural architecture search method with feature selection for PolSAR image classification," *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, vol. 15, pp. 9339–9354, 2022.